

Error detection and correction using Hamming Code.

AIM:

To write a program to implement error detection using HAMMING Code.

Error Correction at Data link layer:

Hamming code is a set of error-correction codes that can be used to detect and correct the errors.

Create sender program with below features:

1. Input to sender file should be a text of any length. Program should convert text to binary.
2. Apply hamming Code Concept on the binary data and add redundant bits to it.
3. save the output in a file called Channel.

Create a receiver program with below features.

1. Receiver program should read the input from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error, display the position of error.
4. Else remove the redundant bits and convert the binary data to ascii and display the output.

CODE :

```
def calc_redundant_bits(m):
```

```
    for r in range(m+1):
```

```
        if (2**r) >= (m+r+1):
```

```
            return r
```

```
def main():
```

```
    data = input("Enter data bits: ").strip()
```

```
    m = len(data)
```

```
    r = calc_redundant_bits(m)
```

```
    n = m+r
```

```
    print(f"Number of r needed : {r}")
```

```
    print(f"Total frame length : {n}")
```

```
    Code = ['0']*(n+1)
```

```
    j = 0
```

```
    for i in range(1, n+1):
```

```
        if (i & (i-1)) != 0:
```

```
            Code[i] = data[-(j+1)]
```

```
            j += 1
```

```
    for i in range(r):
```

```
        parity_pos = 2**i
```

```
        val = 0
```

```
        for pos in range(1, n+1):
```

```
            if (pos & parity_pos) != 0
```

```
                and pos != parity_pos:
```

```
                    if Code[pos] == '1':
```

```
                        val ^= 1
```

```
        Code[parity_pos] = str(val)
```

```
transmitted_str = " ".join(Code[1:][:n])
```

```
print(f"Data to be transmitted :
```

```
    {transmitted_str}")
```

```
Change = input("\n Transmitted bit change")
```

```
    .strip().upper()
```

```

if change == "Yes":
    received_str = input(f"Enter Corrupted data: ")
    if len(received_str) != n:
        print(f"error: received data is exactly {n} bits long!")
        return
    received_code = ['0'] + list(received_str[:n])
    else:
        received_code = list(code)
    error_position = 0
    for i in range(r):
        parity_pos = 2**i
        val = 0
        for pos in range(1, n+1):
            if (pos & parity_pos) != 0:
                if (received_code[pos]) == '1':
                    val += 1
        if val != 0:
            error_position += parity_pos
    if error_position == 0:
        print("\n verification Result: No error detected")
    else:
        print("\n verification Result: Error detected")
        print(f"wrong bit at position: {error_position}")
        received_code[error_position] = '0' if received_code[error_position] == '1' else '1'
        corrected_str = "".join(received_code[1:])
        print(f"Corrected data streams: {corrected_str}")

```

```

if name == "__main__":
    main()

```

output:

① Enter data bits : 101100
Number of redundant bits (r) needed : 4
Total frame length ($m+r$) : 10
Data to be transmitted : 1011100011
Transmitted bit change ? (Yes/No) : Yes
Enter the Corrupted data stream (10 mins):
1011101011
Verification Result: Error detected!
Wrong bit is at position : 4
Corrected data streams : 1011100011

② Enter data bits : 1011
Number of redundant bits (r) needed : 3
Total data frame length ($m+r$) : 7
Data to be transmitted : 1010101
Transmitted bit change ? (Yes/No) : No
Verification Result: No error detected
in transmission

Result:

Thus the Hamming code was implemented successfully for error detection and correction.